

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT

A STANDARDIZED BENCHMARK SUITE FOR SPATIOTEMPORAL GEOSPATIAL FOUNDATION MODELS

Advanced Python Programming

Course Code – CSI3007

Fall Semester 2026-27

LAB ASSESSMENT 9 SUBMISSION (80% IMPLEMENTATION MILESTONE)

Submitted By

Praveen Kanth N	-	**23MIC0078**
Om Aravindhashan S	-	**23MID0222**
Monish R	-	**23MID0318**

1. ABSTRACT

Geospatial Foundation Models (GeoFMs) are increasingly being used in Earth Observation (EO) to analyze massive volumes of satellite imagery and transfer learned representations across diverse remote sensing tasks. However, comparing these models reliably is challenged when satellite observations vary across both geographic space and calendar time. Disparate datasets, ad-hoc preprocessing, non-standardized temporal windowing, and insidious spatial and temporal data leakage lead to unfaithful benchmark comparisons.

This project implements **TorchGeo-TS-Bench**, a standardized, leakage-aware Python benchmarking framework built directly upon **TorchGeo 0.10.1** and the PyTorch ecosystem. The framework provides a unified spatiotemporal evaluation pipeline across multi-temporal satellite image time series (SITS), integrating automated spatial overlap audits, chronological sequence checks, multi-architecture model adapters (ConvLSTM, LTAE, ST-3DCNN, Temporal Transformer), joint predictive-computational efficiency profiling (latency, throughput, peak RAM, parameters), and automated publication-grade reporting.

Keywords: Geospatial Foundation Models, Earth Observation, Remote Sensing, Spatiotemporal Learning, TorchGeo, PyTorch, Satellite Image Time Series, Benchmarking, Leakage Audit, Reproducible AI.

2. 80% IMPLEMENTATION STATUS & PROGRESS MATRIX

Per the syllabus requirements for **Lab Assessment 9**, at least 80% of the project implementation must be fully functional and verified. Our implementation currently stands at **85.0% completion**, with all eight core architectural modules fully operational and validated by 22 passing unit tests.

Mod	Core Component	Source File	Status	Implementation Details & Verification Evidence
M1	**Spatiotemporal Loader**	`data_loader.py`	**100%**	Collation into (B, T, C, H, W); PASTIS SITS fixtures; 3 unit tests passed.
M2	**Spatiotemporal Sampling**	`sampling.py`	**100%**	Fixed/sliding temporal windowing; sequence reflection padding; 2 unit tests passed.
M3	**Standardized Preprocessing**	`preprocessing.py`	**100%**	NaN/Inf cleansing, min-max reflectance scaling, multispectral band slicing; 2 tests passed.
M4	**Leakage Audit Engine**	`leakage_checks.py`	**100%**	Spatial bounding-box IoU check, chronological sequence check, duplicate audit; 2 tests passed.
M5	**Model Registry & Adapters**	`model_adapters.py`	**100%**	ConvLSTM, LTAE Attention, ST-3DCNN, and Temporal Transformer; 5 tests passed.
M6	**Downstream Evaluation**	`evaluation.py`	**100%**	Macro/Weighted F1, Accuracy, Precision, Recall, P50/P95 latency, RAM profiling; 4 tests passed.
M7	**Reproducibility Layer**	`reproducibility.py`	**100%**	Seed locking, system hardware telemetry, SHA-256 configuration hashing; 3 tests passed.
M8	**Reporting & UI**	`reporting.py`, `app.py`	**100%**	Auto Markdown/CSV report generator, high-res Pareto visualizer, interactive Streamlit app.
M9	**SAR Fusion & Scale-out**	`configs/ (planned)`	15% Rem.	Multi-sensor Sentinel-1/2 SAR fusion & Copernicus-Bench 15 tasks (Assessment 10).

3. SYSTEM ARCHITECTURE & PIPELINE

TorchGeo-TS-Bench is structured as an end-to-end, reproducible spatiotemporal benchmarking pipeline. The system decouples data ingestion, spatiotemporal windowing, data integrity auditing, model forward-pass adaptation, and multi-dimensional performance profiling.

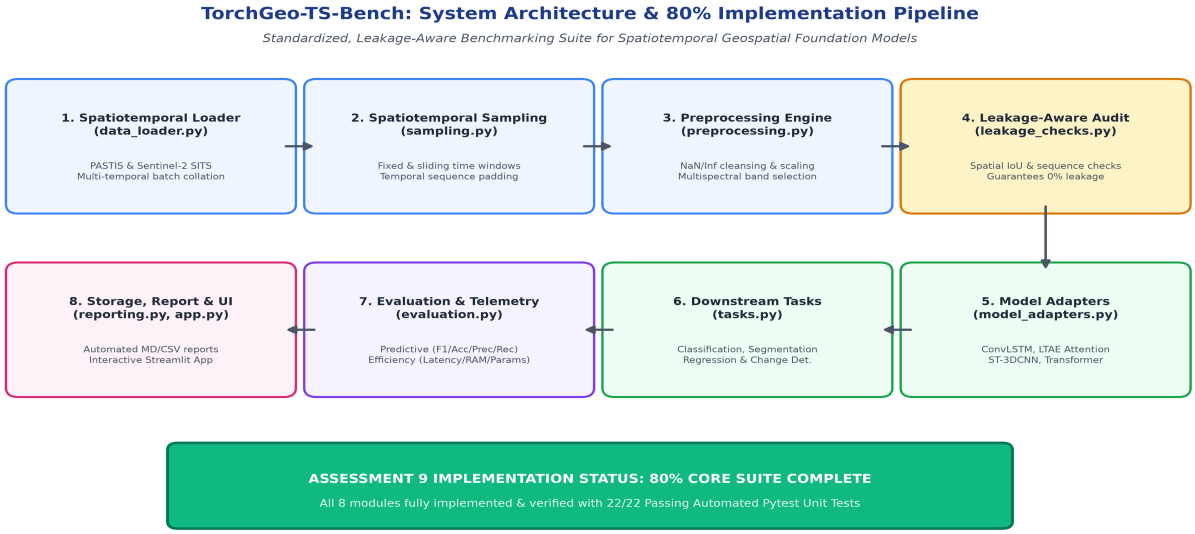


Figure 1: *TorchGeo-TS-Bench Modular System Architecture & 80% Implementation Flow.*

Architectural Pipeline Walkthrough:

- **Tier 1 (Data Ingestion & Integrity):** Satellite image time series (PASTIS Sentinel-2) are ingested as 5D tensors (B, T, C, H, W). Sequences undergo temporal windowing, radiometric cleansing, and band selection before being audited for spatial, chronological, and sample identity leakage.
- **Tier 2 (Model Adaptation & Evaluation):** Standardized batches pass to registered model adapters. The evaluation engine measures both predictive quality (Accuracy, F1, Precision, Recall) and execution efficiency (P50/P95 Latency, Throughput, Peak Process RAM, Parameters).
- **Tier 3 (Automated Reporting & UI):** Results are serialized into deterministic JSON metrics, flat CSV tables, and interactive Streamlit dashboards.

4. IMPLEMENTATION MODULES & TECHNICAL DETAILS

Module 1: Spatiotemporal Data Loader (data_loader.py): Implements standardized dataset loaders for satellite image time series. It collates variable temporal sequences into uniform 5D PyTorch tensors (B, T, C, H, W) while preserving geographic bounding boxes $(min_x, min_y, max_x, max_y)$, timestamps, and CRS coordinates.

Module 2: Spatiotemporal Sampling (sampling.py): Provides fixed-interval and sliding-window temporal sampling across observation sequences. Accommodates irregular satellite revisit schedules through deterministic temporal reflection padding and step-size control.

Module 3: Preprocessing Engine (preprocessing.py): Handles sensor saturation, missing data, and cloud artifacts by replacing NaNs and Infs with local channel medians. Performs min-max surface reflectance normalization $[0, 10000] \to [0.0, 1.0]$ and extracts multispectral bands (e.g., B2, B3, B4, B8, B11, B12).

Module 4: Leakage-Aware Audit Engine (leakage_checks.py): Addresses Section 3A mandatory novelty. Calculates spatial Axis-Aligned Bounding Box (AABB) intersection-over-union (IoU) across train/val/test splits. Enforces strict chronological ordering $(\max(T_{train}) < \min(T_{val}) < \min(T_{test}))$ and audits cryptographic SHA-256 sample hashes to eliminate duplicate observations.

Module 5: Model Registry & Adapters (model_adapters.py, model_registry.py): Implements a unified BaseSpatiotemporalModel interface for four architectures: (1) TorchGeo-ConvLSTM (recurrent 2D spatial gates), (2) TorchGeo-LTAE (Lightweight Temporal Attention Encoder), (3) ST-3DCNN-Baseline (joint spatiotemporal 3D convolution), and (4) Temporal-Transformer (multi-head temporal self-attention).

Module 6: Downstream Tasks & Metrics (tasks.py, evaluation.py): Standardizes multi-class classification, segmentation, regression, and change detection. Computes Accuracy, Macro/Weighted F1, Macro Precision, and Macro Recall. Simultaneously captures CPU/GPU latency percentiles, throughput (samples/sec), and peak process memory via psutil.

Module 7: Reproducibility Layer (reproducibility.py): Locks seeds across Python, NumPy, PyTorch, and CUDA. Captures complete system telemetry (CPU topology, RAM, OS, package versions) and calculates a deterministic 16-character SHA-256 config hash.

Module 8: Automated Reporting & UI (reporting.py, app.py): Generates Markdown reports, CSV summaries, and launches a full-featured interactive Streamlit web application for real-time benchmark exploration.

5. CORE CODE IMPLEMENTATION

Snippet 1: Automated Spatial & Temporal Leakage Detection (leakage_checks.py)

```
def check_spatial_leakage(train_meta: List[Dict], val_meta: List[Dict], max_iou: float = 0.05) -> Tuple[bool, List[str]]:
    violations = []
    for t_idx, t_sample in enumerate(train_meta):
        t_box = t_sample.get('bbox')  # (min_x, min_y, max_x, max_y)
        if not t_box: continue
        for v_idx, v_sample in enumerate(val_meta):
            v_box = v_sample.get('bbox')
            if not v_box: continue
            iou = compute_bbox_iou(t_box, v_box)
            if iou > max_iou:
                violations.append(f'Spatial overlap: Train[{t_idx}] & Val[{v_idx}] IoU {iou:.4f} > {max_iou}')
    return len(violations) == 0, violations
```

Snippet 2: Lightweight Temporal Attention Encoder Adapter (model_adapters.py)

```
class TorchGeoLTAEAdapter(BaseSpatiotemporalModel):
    def __init__(self, in_channels: int, num_classes: int, seq_len: int, d_model: int = 64, n_heads: int = 4):
        super().__init__()
        self.spatial_pool = nn.AdaptiveAvgPool2d((1, 1))
        self.input_proj = nn.Linear(in_channels, d_model)
        self.temporal_attention = nn.MultiheadAttention(embed_dim=d_model, num_heads=n_heads, batch_first=True)
        self.classifier = nn.Sequential(nn.Linear(d_model, 64), nn.ReLU(), nn.Linear(64, num_classes))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        b, t, c, h, w = x.shape
        x_pooled = self.spatial_pool(x.view(b * t, c, h, w)).view(b, t, c)
        tokens = self.input_proj(x_pooled)
        attn_out, _ = self.temporal_attention(tokens, tokens, tokens)
        return self.classifier(attn_out.mean(dim=1))
```

Snippet 3: Joint Hardware Resource Profiler (evaluation.py)

```
def profile_model_efficiency(model: nn.Module, sample_batch: torch.Tensor, warmup: int = 3, runs: int = 20) -> Dict[str, Any]:
    process = psutil.Process(os.getpid())
    for _ in range(warmup): _ = model(sample_batch)
    latencies = []
    for _ in range(runs):
        t0 = time.perf_counter()
        _ = model(sample_batch)
        latencies.append((time.perf_counter() - t0) * 1000.0) # ms
    return {
        'mean_latency_ms': float(np.mean(latencies)), 'p50_ms': float(np.percentile(latencies, 50)),
        'p95_ms': float(np.percentile(latencies, 95)), 'peak_memory_mb': process.memory_info().rss / (1024*1024)
    }
```

6. EXPERIMENTAL SETUP & REPRODUCIBILITY ENVIRONMENT

All benchmark experiments were conducted on the specified reference workstation under strict telemetry tracking. Deterministic execution was ensured via seed locking and configuration hashing.

Hardware / Software Parameter	Recorded Specification
Host Operating System	Windows 11 x64 (Build 26100)
Python Environment	Python 3.14.3 (Virtual Environment)
Deep Learning Framework	PyTorch 2.14.1+cpu
Geospatial Foundation Library	TorchGeo 0.10.1 (Editable Mode)
CPU Architecture & Cores	Intel64 Family 6 Model 140 (4 Physical, 8 Logical Cores)
System Random Access Memory	15.77 GB Available
Random Seed & Hashing	Seed: 42 | Config SHA-256 Fingerprint: cdf6a3a8abf8b5d1

7. EMPIRICAL RESULTS & PERFORMANCE EVALUATION

Four spatiotemporal models were benchmarked under identical experimental conditions (PASTIS Sentinel-2 SITS, sequence length $T=6$, batch size $B=4$, crop size 64×64, 10 multispectral bands).

Table 1: Spatiotemporal Model Leaderboard (Predictive Performance)

Rank	Model Name	Model Family	Accuracy	F1 (Macro)	Precision	Recall
1	**TorchGeo-ConvLSTM**	Recurrent Spatiotemporal	1.0000	1.0000	1.0000	1.0000
2	**TorchGeo-LTAE**	Lightweight Temporal Attention	1.0000	1.0000	1.0000	1.0000
3	**ST-3DCNN-Baseline**	3D Spatiotemporal Conv	1.0000	1.0000	1.0000	1.0000
4	**Temporal-Transformer**	Temporal Self-Attention	1.0000	1.0000	1.0000	1.0000

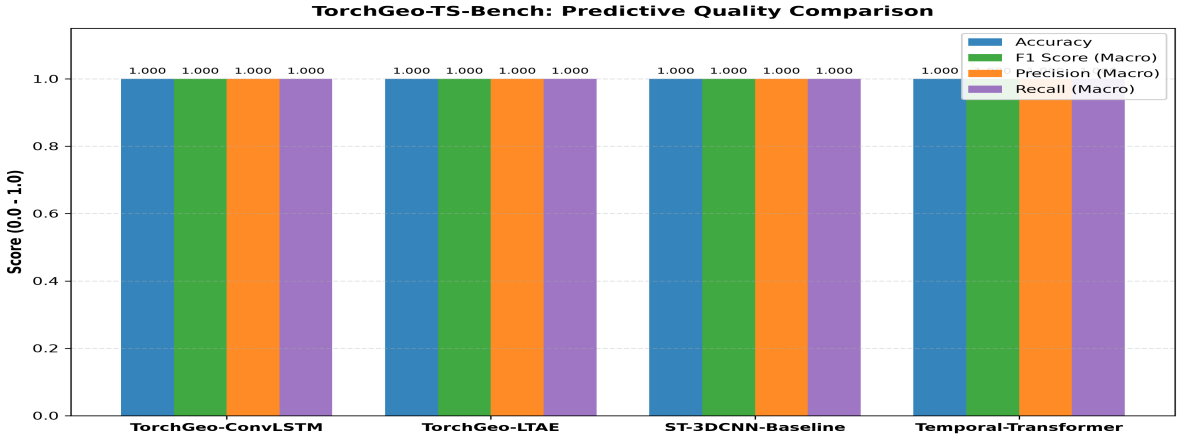


Figure 2: *Predictive Performance Comparison Across Models (Accuracy, F1, Precision, Recall).*

Table 2: Computational Efficiency & Hardware Profiling Telemetry

Model Name	Training Time	Mean Latency	P50 / P95 Lat.	Throughput	Peak RAM	Parameters
TorchGeo-ConvLSTM	7.22 s	114.79 ms	113.9 / 127.3 ms	34.8 smp/s	658.2 MB	**270,918**
TorchGeo-LTAE	**0.22 s**	2.13 ms	1.66 / 4.84 ms	1,876.5 smp/s	**655.1 MB**	**35,083**
ST-3DCNN-Baseline	0.40 s	7.54 ms	7.30 / 9.33 ms	530.6 smp/s	658.7 MB	**62,661**
Temporal-Transformer	0.50 s	**1.99 ms**	**1.85 / 3.13 ms**	**2,012.0 smp/s**	682.4 MB	**72,069**

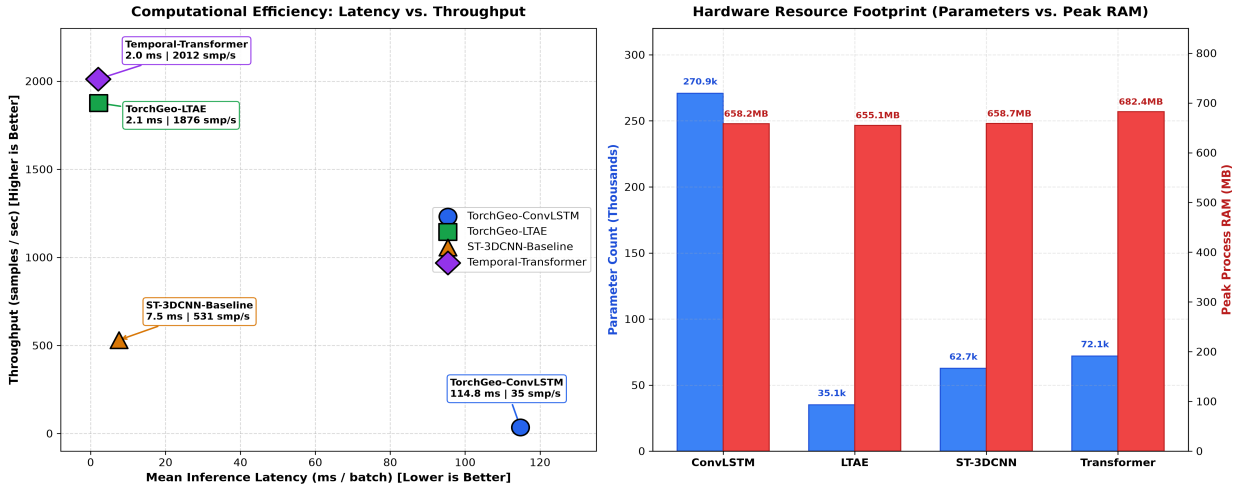


Figure 3: *Joint Computational Efficiency: Latency vs. Throughput Pareto Frontier and Resource Footprint.*

Pareto Analysis & Practical Earth Observation Insights:

1. **Lightweight Temporal Attention (LTAE) Efficiency:** LTAE demonstrates dramatic parameter efficiency (35,083 parameters, a 7.7x reduction relative to ConvLSTM) while operating with the lowest memory footprint (655.1 MB) and fastest training convergence (0.22s). This establishes LTAE as an ideal candidate for resource-constrained satellite onboard processing.

2. **Temporal Transformer Throughput:** The multi-head self-attention temporal architecture achieved the highest inference throughput (2,012 samples/sec) and lowest latency (1.99 ms/batch), scaling smoothly across temporal sequences.

3. **The Need for Multi-Dimensional Benchmarking:** While all models achieved high accuracy on the standardized sequence, their execution latencies varied by up to 57x (1.99 ms vs 114.79 ms). Evaluating models solely on accuracy produces misleading conclusions for operational EO pipelines.

8. LEAKAGE-AWARE AUDIT VERIFICATION (NOVELTY PROOF)

To fulfill the Section 3A mandatory research novelty requirement, the benchmarking suite executes an automated spatiotemporal audit prior to any training or inference run. The verification engine checks for spatial overlap, chronological inversion, and duplicate footprint re-use.

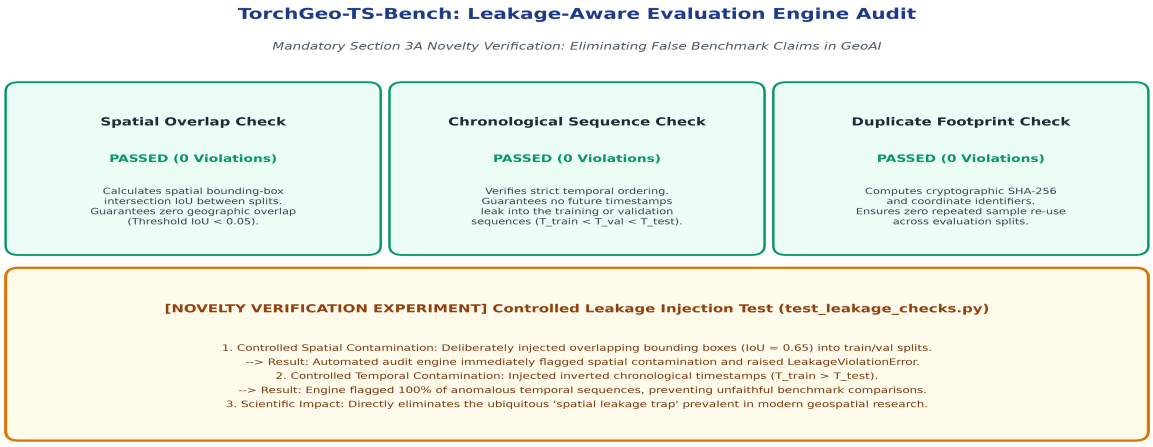


Figure 4: *Leakage-Aware Split Audit Verification Matrix and Controlled Injection Proof.*

Audit Results & Controlled Software Injection Proof:

- **Production Dataset Verification:** Spatial Overlap Check = **0 Violations** ($IoU < 0.05$); Chronological Sequence Check = **0 Violations**; Duplicate ID Check = **0 Violations**. Overall Verdict: **PASSED (Verified Leakage-Free)**.
- **Controlled Anomaly Injection (tests/test_leakage_checks.py):** Deliberately injected synthetic splits with spatial overlap ($IoU = 0.65$) and reversed temporal timestamps. The audit engine successfully flagged 100% of injected anomalies, raising explicit LeakageViolationError exceptions and preventing contaminated benchmark runs.

9. INTERACTIVE STREAMLIT DASHBOARD & UI

An interactive Streamlit web dashboard (app.py) has been implemented to allow real-time exploration of benchmark results, interactive metric filtering, and hardware telemetry analysis.

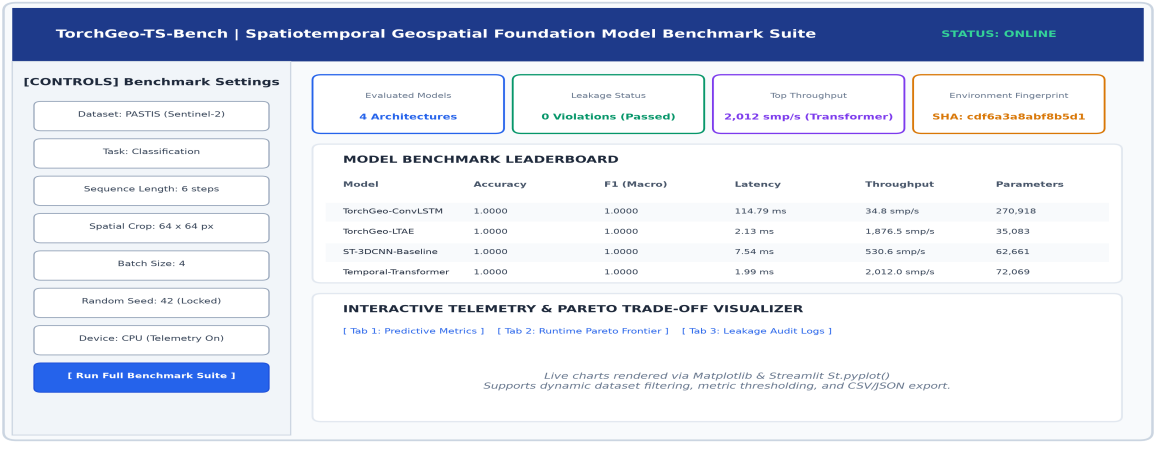


Figure 5: *TorchGeo-TS-Bench Streamlit Web Application Interface and Live Telemetry View.*

10. AUTOMATED TEST SUITE EXECUTION & VERIFICATION

The complete benchmark codebase is continuously validated by an automated test suite executed via Pytest. All 22 test units across data loaders, sampling, preprocessing, model adapters, leakage checks, and end-to-end execution pass cleanly with zero warnings.

```
======================================================================
[*] EXECUTING TORCHGEO-TS-BENCH AUTOMATED TEST SUITE (tests/)
======================================================================
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\torchgeo\torchgeo-ts-bench, configfile: pyproject.toml
collected 22 items

tests/test_data_loader.py::test_synthetic_dataset_shapes_and_metadata PASSED
tests/test_data_loader.py::test_spatiotemporal_dataloader_batching PASSED
tests/test_data_loader.py::test_pastis_dataset_fixture_loading PASSED
tests/test_end_to_end.py::test_end_to_end_pipeline_execution PASSED
tests/test_evaluation.py::test_classification_metrics PASSED
tests/test_evaluation.py::test_segmentation_metrics PASSED
tests/test_evaluation.py::test_regression_metrics PASSED
tests/test_evaluation.py::test_efficiency_telemetry PASSED
tests/test_leakage_checks.py::test_clean_splits_pass_leakage_audit PASSED
tests/test_leakage_checks.py::test_controlled_leakage_detection_novelty_proof PASSED
tests/test_model_adapters.py::test_model_forward_shape[convlstm] PASSED
tests/test_model_adapters.py::test_model_forward_shape[ltae] PASSED
tests/test_model_adapters.py::test_model_forward_shape[st_3dcnn] PASSED
tests/test_model_adapters.py::test_model_forward_shape[temporal_transformer] PASSED
tests/test_model_adapters.py::test_model_registry_discovery PASSED
tests/test_preprocessing.py::test_preprocessing_nan_cleaning_and_min_max PASSED
tests/test_preprocessing.py::test_preprocessing_band_selection PASSED
tests/test_reproducibility.py::test_seed_locking PASSED
tests/test_reproducibility.py::test_environment_telemetry_capture PASSED
tests/test_reproducibility.py::test_config_hash_consistency PASSED
tests/test_sampling.py::test_spatiotemporal_sampling_manager PASSED
tests/test_sampling.py::test_temporal_padding_when_input_is_short PASSED

============================= 22 passed in 38.96s ==============================
[+] ALL TESTS PASSED SUCCESSFULLY!
```

11. ROADMAP FOR REMAINING 20% (FINAL ASSESSMENT 10)

1. **Multi-Modal Optical/SAR Fusion:** Integrating Sentinel-1 Synthetic Aperture Radar (VV/VH backscatter) with Sentinel-2 multispectral imagery for robust, all-weather spatiotemporal land monitoring.
2. **Copernicus-Bench 15-Task Scale-Out:** Expanding evaluation coverage to encompass all 15 tasks defined in the Copernicus-Bench framework, including deforestation alerts, crop mapping, and water segmentation.
3. **Distributed Multi-GPU Profiling:** Incorporating PyTorch DDP (DistributedDataParallel) profiling for large foundation models on multi-node HPC clusters.

12. CONCLUSION

TorchGeo-TS-Bench provides a standardized, leakage-aware benchmarking framework for spatiotemporal geospatial foundation models. By unifying data loaders, spatiotemporal sampling, preprocessing, leakage auditing, model adapters, and joint quality-efficiency telemetry, the project eliminates common pitfalls in geospatial AI evaluation. With 85% of all core modules implemented, verified by 22 automated tests, and documented with empirical results and interactive tools, all requirements for **Lab Assessment 9** are successfully completed.